

Lab Session: Vision-Based Target Tracking Using Camera

Course: Mobile Robotics Lab

Platform: TurtleBot3 Waffle (Gazebo Simulation)

1. Lab Objective

This lab introduces image-based perception and control using the onboard camera. In this session, students will use the `/camera/image_raw` topic to detect a colored object and generate motion commands to align the robot and move toward the target.

2. Learning Outcomes

After completing this lab, students will be able to:

- Subscribe to and process image data in ROS 2
- Use OpenCV for feature extraction (color detection)
- Compute control actions based on visual input
- Implement a simple vision-based navigation behavior

3. Pre-Lab Setup

Launch simulation:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Verify camera topic:

```
ros2 topic list
ros2 topic echo /camera/image_raw
```

View camera stream:

```
ros2 run rqt_image_view rqt_image_view
```

4. Required Packages

Install OpenCV and cv_bridge if needed:

```
sudo apt install ros-humble-cv-bridge python3-opencv
```

5. Lab Tasks

Task 1: Subscribe to `/camera/image_raw` and display the image using OpenCV.

Task 2: Convert the image to HSV and perform color segmentation.

Task 3: Compute the centroid of the detected object.

Task 4: Implement proportional control to align the robot with the object.

Task 5: Move toward the object when centered and stop at close range.

6. Motion Commands

Publish velocity commands to `/cmd_vel`.

Use `ros2 topic echo /cmd_vel` to monitor robot motion commands.

7. Suggested Student Exercises

Exercise 1: Change the HSV threshold values to detect a different colored object. Students should compare how lighting and color thresholds affect segmentation quality.

Exercise 2: Experimentally tune the proportional gain (kp). Observe the effect of small and large kp values on robot motion, oscillation, and stability.

Exercise 3: Modify the logic so the robot stops rotating when no object is detected.

Exercise 4: Detect multiple colors and assign different robot behaviors.

8. Key Concepts to Understand

Students should understand the following perception-to-control pipeline:

`/camera/image_raw` → OpenCV Processing → Object Detection → Error Computation → `/cmd_vel`

The lab demonstrates how image perception can be converted into real-time robot motion decisions.

9. Boilerplate ROS 2 Node (Students Fill In)

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
import cv2
import numpy as np

class CameraFollower(Node):

    def __init__(self):
        super().__init__('camera_follower')

        self.subscription = self.create_subscription(
            Image,
```

```

        '/camera/image_raw',
        self.image_callback,
        10)

self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)

self.bridge = CvBridge()

# TODO: Tune control parameters
self.kp = _____
self.center_threshold = _____

def image_callback(self, msg):
    pass

```

10. Execution Commands

Build and run:

```

colcon build
source install/setup.bash
ros2 run <your_package_name> camera_follower

```

11. Expected Demonstration

Students must demonstrate:

- Detection of colored object
- Alignment with object
- Forward motion toward object
- Smooth stopping

12. Deliverables

Students are required to submit:

1. ROS 2 package pushed to GitHub
2. Source code for the vision-based control node
3. Screenshots of camera view and segmented mask
4. rqt_graph showing active nodes and topics
5. Demonstration video of robot behavior
6. Observations on controller tuning and color segmentation
7. Brief conclusion summarizing learning outcomes

13. Conclusion

This lab provided hands-on experience in vision-based perception and control using ROS 2 and OpenCV. Students implemented image processing, feature extraction, and closed-loop control to generate robot motion from camera data. The exercise demonstrated how perception, error computation, and motion control are integrated in mobile robotic systems.